

LAB 05 – PC - IT24610786

Exercise 1 -OpenMP Barrier

Barrier Synchronization: The `#pragma omp barrier` directive is used to synchronize all threads within an OpenMP parallel region.

Each thread must reach the barrier before any thread is allowed to continue executing the statements after it.

In this exercise, two threads were created using `OMP_NUM_THREADS=2`.

Both threads printed their thread identification before reaching the barrier.

After all threads reached the barrier, they continued to execute the statements following the barrier.

The experiment demonstrates how barriers can be used to divide parallel execution into synchronized phases.

```
ubuntu@ip-172-31-30-61:~/PC-Lab05$ gcc -fopenmp exercisel.c -o exercisel
ubuntu@ip-172-31-30-61:~/PC-Lab05$ OMP_NUM_THREADS=2 ./exercisel
thread # 0
thread # 1
Outside Barrier # 0
Hi again from thread # 0
Outside Barrier # 1
Hi again from thread # 1
ubuntu@ip-172-31-30-61:~/PC-Lab05$ |
```

Exercise 2 — OpenMP single

Single Directive: The `#pragma omp single` directive specifies that a block of code should be executed by only one thread within the parallel region.

In this exercise, two threads were created.

Thread 0 executed the statements inside the single region, while thread 1 did not execute that block.

The thread that executes a single region is not necessarily predetermined; any participating thread may execute it.

By default, an implicit barrier exists at the end of the single region, causing the other threads to wait before continuing.

```

ubuntu@ip-172-31-30-61:~$ nano exercise2.c
ubuntu@ip-172-31-30-61:~$ gcc -fopenmp exercise2.c -o exercise2
ubuntu@ip-172-31-30-61:~$ OMP_NUM_THREADS=2 ./exercise2
Before Single - thread # 0
Inside Single - thread # 0
Before Single - thread # 1
After Single - thread # 0
After Single - thread # 1
ubuntu@ip-172-31-30-61:~$ |

```

Exercise 3 — OpenMP master

Master Directive: The `#pragma omp master` directive specifies that only the master thread, which is thread 0, can execute the associated block of code.

In this exercise, thread 0 executed the statements inside the master region, while thread 1 skipped the region.

Unlike the `single` directive, the `master` directive does not provide an implicit barrier at the end of the region.

Therefore, other threads are not required to wait for the master thread to finish before continuing.

```

ubuntu@ip-172-31-30-61:~$ nano exercise3.c
ubuntu@ip-172-31-30-61:~$ gcc -fopenmp exercise3.c -o exercise3
ubuntu@ip-172-31-30-61:~$ OMP_NUM_THREADS=2 ./exercise3
Before Master - thread # 0
Inside Master - thread # 0
After Master - thread # 0
Before Master - thread # 1
After Master - thread # 1
ubuntu@ip-172-31-30-61:~$ |

```

Exercise 4 — Nested parallel for

Nested Loop Parallelization: In the first implementation, `#pragma omp parallel for` was applied only to the outer loop.

Therefore, OpenMP distributed the five iterations of the outer loop among the available threads, while each thread completed all ten iterations of the inner loop for its assigned outer-loop values.

With two threads, thread 0 processed $r=0,1,2$, while thread 1 processed $r=3,4$.

The second implementation used `#pragma omp parallel for collapse(2)`.

The `collapse(2)` clause combines the two nested loops into a single iteration space containing $5 \times 10 = 50$ iterations.

These iterations can then be distributed more finely among the threads.

The output demonstrated this because the same r value could be processed by different threads, such as $r=2$, where thread 0 processed $k=0-4$ and thread 1 processed $k=5-9$.

This provides greater parallelism when the outer loop has a small number of iterations.

```
ubuntu@ip-172-31-30-61:~$ nano exercise4.c
ubuntu@ip-172-31-30-61:~$ gcc -fopenmp exercise4.c -o exercise4
ubuntu@ip-172-31-30-61:~$ OMP_NUM_THREADS=2 ./exercise4
Thread 0: r=0 k=0
Thread 0: r=0 k=1
Thread 0: r=0 k=2
Thread 0: r=0 k=3
Thread 0: r=0 k=4
Thread 0: r=0 k=5
Thread 0: r=0 k=6
Thread 0: r=0 k=7
Thread 0: r=0 k=8
Thread 0: r=0 k=9
Thread 0: r=1 k=0
Thread 0: r=1 k=1
Thread 0: r=1 k=2
Thread 0: r=1 k=3
Thread 0: r=1 k=4
Thread 0: r=1 k=5
Thread 0: r=1 k=6
Thread 0: r=1 k=7
Thread 0: r=1 k=8
Thread 0: r=1 k=9
Thread 0: r=2 k=0
Thread 0: r=2 k=1
Thread 0: r=2 k=2
Thread 0: r=2 k=3
Thread 0: r=2 k=4
Thread 0: r=2 k=5
Thread 0: r=2 k=6
Thread 0: r=2 k=7
Thread 0: r=2 k=8
Thread 0: r=2 k=9
Thread 1: r=3 k=0
Thread 1: r=3 k=1
Thread 1: r=3 k=2
Thread 1: r=3 k=3
Thread 1: r=3 k=4
Thread 1: r=3 k=5
Thread 1: r=3 k=6
Thread 1: r=3 k=7
Thread 1: r=3 k=8
Thread 1: r=3 k=9
```

```
Thread 1: r=3 k=7
Thread 1: r=3 k=8
Thread 1: r=3 k=9
Thread 1: r=4 k=0
Thread 1: r=4 k=1
Thread 1: r=4 k=2
Thread 1: r=4 k=3
Thread 1: r=4 k=4
Thread 1: r=4 k=5
Thread 1: r=4 k=6
Thread 1: r=4 k=7
Thread 1: r=4 k=8
Thread 1: r=4 k=9
ubuntu@ip-172-31-30-61:~$ |
```

Exercise 5 — Recursive Fibonacci using OpenMP Tasks

The recursive Fibonacci function was parallelized using OpenMP tasks. For each recursive call, two independent tasks were created to calculate $\text{fib}(n-1)$ and $\text{fib}(n-2)$.

Since these calculations are independent, they can be executed concurrently by different OpenMP threads.

The single directive was used to ensure that only one thread initially creates the recursive task structure, while the taskwait directive ensures that both child tasks have completed before their results are added.

The program was executed using two OpenMP threads and produced $\text{fib}(20) = 6765$, confirming the correctness of the parallel implementation.

```
ubuntu@ip-172-31-30-61:~$ nano exercise5.c
ubuntu@ip-172-31-30-61:~$ gcc -fopenmp exercise5.c -o exercise5
ubuntu@ip-172-31-30-61:~$ OMP_NUM_THREADS=2 ./exercise5
fib(20) = 6765
ubuntu@ip-172-31-30-61:~$ |
```

Exercise 6 — Element-wise Multiplication

Strip Mining and SIMD Parallelization: The element-wise multiplication of two arrays containing 1,000,000 elements was parallelized using OpenMP.

Strip mining was applied by dividing the arrays into smaller strips of 64 elements.

The outer loop distributes these strips among OpenMP threads using parallel for, while the inner loop processes the elements within each strip.

The simd directive allows the compiler to vectorize the inner loop so that multiple elements can potentially be processed simultaneously using SIMD instructions.

A strip size of 64 was selected because it is compatible with common SIMD vector widths.

The program produced the expected results, with $C[999999] = 1999998.00$, confirming the correctness of the implementation.

The measured execution time using two threads was approximately 0.004594 seconds.

```
ubuntu@ip-172-31-30-61:~$ nano exercise6.c
ubuntu@ip-172-31-30-61:~$ gcc -O3 -march=native -fopenmp exercise6.c -o exercise6
ubuntu@ip-172-31-30-61:~$ OMP_NUM_THREADS=2 ./exercise6
C[0] = 0.00
C[999999] = 1999998.00
Time = 0.004594 seconds
ubuntu@ip-172-31-30-61:~$ |
```